Selected-Entry Matrix Multiplication, APSP, and Integer 3SUM

Trevor Vaughn

Contents

1 Selected-Entry Matrix Multiplication **1**
1.1 Sharing and Recovering Block Products 1
1.2 Recursion and Running Time 3

2 All-Pairs Shortest Paths (APSP) **5**

3 Integer 3SUM **8**

Relation to prior work. The selected-entry matrix multiplication argument modifies the simplified construction of Karingula and Lovett [2]. This note synthesizes that modified construction, a simpler, direct reduction of APSP to selected-entry multiplication, and the integer 3SUM work of Chan, Itakura, Ma, and Yu [1].

1 Selected-Entry Matrix Multiplication

Let $X \in \mathbb{Z}^{n\times d}$ and $Y \in \mathbb{Z}^{d\times m}$. We want only w specified entries of XY. Computing each dot product separately takes $O(wd)$ operations, while computing the full product may waste most of its work. We instead recursively multiply smaller matrices, arranging for different outputs to reuse the same smaller products. Only entries needed by the requested outputs are evaluated recursively.

Theorem 1 (Selected-entry matrix multiplication)**.** *For every constant $0 < \tau \le 1$, there is an integer $\kappa = O(\tau^{-2})$ such that, if $d^\kappa \le \min\{n, m\}$, any $w \ge 1$ distinct specified entries of XY can be computed deterministically in time*

$$\widetilde{O}\big((n+m)d^\kappa + d^\tau\sqrt{nmw}\big)\,. \tag{1}$$

The input entries are integers of magnitude polynomial in $n + m$.

The main step is a block multiplication identity. Splitting the middle dimension into four parts expresses each output block as a sum of four smaller products, so computing RS output blocks separately would use $4RS$ products. We replace these with one private correction per output and a pool of products shared by many outputs.

The sharing pattern comes from a grid: each output is a coordinate line, and each grid point represents one product shared by all lines through it. At a point, we add the left factors contributed by those lines, add their right factors, and multiply the sums. Of course this creates cross terms. The key is that, when viewed along any fixed line, the unwanted left and right factors vary independently. A simple checksum padding then cancels the cross terms using one additional product. Since the resulting identity holds entrywise, we can recurse only on the entries actually requested.

1.1 Sharing and Recovering Block Products

The grid and its output addresses. Consider the k-dimensional grid $[9]^k$. Regard each coordinate line (obtained by varying one coordinate and fixing the others) as an output block, and each grid point as a shared multiplication. Every line has nine points, but every point belongs to k lines. Hence $k9^{k-1}$ outputs

use just 9^k shared products, or $9/k$ per output. For $k = 2$, these are the rows, columns, and intersections of a 9×9 grid. Increasing the dimension increases the number of outputs that can reuse the product at any point.

To make this incidence pattern compatible with matrix multiplication, regard each grid coordinate as a pair in $[3]^2$ and write

$$z = ((a_1, b_1), \ldots, (a_k, b_k)), \qquad a_h, b_h \in [3].$$

A line in direction h fixes all pairs except (a_h, b_h); let a_{-h} and b_{-h} denote the tuples of the remaining first and second coordinates. Assign that line the output-matrix address

$$\underbrace{(h, a_{-h})}_{\text{output row}}, \qquad \underbrace{b_{-h}}_{\text{output column}}. \tag{2}$$

These addresses give $R = k3^{k-1}$ possible rows and $S = 3^{k-1}$ columns, and thus identify all $RS = k9^{k-1}$ output blocks with the coordinate lines. The nine points on a line, obtained by varying $(a_h, b_h) = (i, j)$, will provide the nine terms indexed by $(i, j) \in [3]^2$.

The asymmetric labeling is deliberate: a left input belongs to a matrix row, governed by the a-coordinates, while a right input belongs to a matrix column, governed by the b-coordinates. This is what will make the interference from other lines manageable.

What we multiply at a grid point. Take a block product $C = UV$, with R block rows and four block columns in U, and four block rows and S block columns in V. Every output block is

$$C_{r,c} = \sum_{\ell=1}^{4} U_{r,\ell} V_{\ell,c}.$$

We will encode the four left blocks of each row r in a 3×3 table $P_r(i, j)$ and the four right blocks of each column c in a table $Q_c(i, j)$. The precise padding will be specified when we need to recover the outputs. For now, note that P_r depends only on matrix row r, and Q_c only on matrix column c.

At a grid point z, one line passes in each direction h. That line uses entry (a_h, b_h) from the two tables associated with its output. We pool all k contributions into a single smaller matrix product:

$$\begin{aligned} A_z &= \sum_{h=1}^{k} P_{(h,a_{-h})}(a_h, b_h), \\ B_z &= \sum_{h=1}^{k} Q_{b_{-h}}(a_h, b_h), \qquad W_z = A_z B_z. \end{aligned} \tag{3}$$

The same W_z is used by all k output lines passing through z. Its factors contain contributions from different outputs, so expanding W_z produces unwanted cross terms. What makes the construction work is the form these terms take when we follow one line through its nine points.

Why the interference separates. The case $k = 2$ shows the mechanism. Fix a line in direction 1, with $(a_2, b_2) = (u, v)$. Its nine grid points are $z^{ij} = ((i, j), (u, v))$, and the pooled product at each point is

$$W_{z^{ij}} = \big(P_{(1,u)}(i, j) + P_{(2,i)}(u, v)\big)\big(Q_v(i, j) + Q_j(u, v)\big). \tag{4}$$

The first terms are the factors intended for our output. The foreign left term $P_{(2,i)}(u, v)$ varies only with i, whereas the foreign right term $Q_j(u, v)$ varies only with j.

The same observation holds for any k. Fix a line in direction h, with output row $r = (h, a_{-h})$ and column $c = b_{-h}$, and write z^{ij} for its point with $(a_h, b_h) = (i, j)$. Our line contributes $P_r(i, j)$ and $Q_c(i, j)$ there. For every other direction $\ell \neq h$, its table position (a_ℓ, b_ℓ) stays fixed as (i, j) varies. Its left table is indexed by $(\ell, a_{-\ell})$, which can change with i but not j; its right table is indexed by $b_{-\ell}$, which can change with j but not i. Summing all foreign contributions gives

$$A_{z^{ij}} = P_r(i, j) + x_i, \qquad B_{z^{ij}} = Q_c(i, j) + y_j \tag{5}$$

for some blocks x_i, y_j. Thus the entire interference is *separable*: the unwanted left factors depend only on the first coordinate of the slot and the unwanted right factors only on the second. We next choose the padding so that this special interference can be removed with one correction product.

Checksums and the decoder. To recover $C_{r,c}$, we want the mixed terms to vanish when the nine products in (5) are added. It suffices to make every column of P_r and every row of Q_c sum to zero. A zero last column of P_r and zero last row of Q_c will also reveal the unwanted factors x_i, y_j, allowing us to subtract what remains. Both properties follow by adjoining checksums to the four original blocks.

Let $p_{11}, p_{12}, p_{21}, p_{22}$ be the four left blocks from row r of U, in order, and let $q_{11}, q_{12}, q_{21}, q_{22}$ be the four right blocks from column c of V. Set

$$P_r = \begin{pmatrix} p_{11} & p_{12} & 0 \\ p_{21} & p_{22} & 0 \\ -p_{11}-p_{21} & -p_{12}-p_{22} & 0 \end{pmatrix}, \qquad Q_c = \begin{pmatrix} q_{11} & q_{12} & -q_{11}-q_{12} \\ q_{21} & q_{22} & -q_{21}-q_{22} \\ 0 & 0 & 0 \end{pmatrix}. \tag{6}$$

Each new entry of P_r is paired with a zero entry of Q_c (or vice versa), so the desired output is still

$$C_{r,c} = \sum_{i,j\in[3]} P_r(i,j)Q_c(i,j). \tag{7}$$

By construction,

$$\sum_i P_r(i,j) = 0, \qquad \sum_j Q_c(i,j) = 0, \qquad P_r(i,3) = Q_c(3,j) = 0. \tag{8}$$

For our fixed line, expand the nine shared products. By (5) and the zero-sum conditions,

$$\begin{aligned} \sum_{i,j\in[3]} W_{z^{ij}} &= \sum_{i,j}(P_r(i,j)+x_i)(Q_c(i,j)+y_j) \\ &= C_{r,c} + \sum_j \Big(\sum_i P_r(i,j)\Big) y_j + \sum_i x_i \Big(\sum_j Q_c(i,j)\Big) \\ &\quad + \Big(\sum_i x_i\Big)\Big(\sum_j y_j\Big) \\ &= C_{r,c} + \Big(\sum_i x_i\Big)\Big(\sum_j y_j\Big). \end{aligned}$$

The two mixed sums vanish, and the remaining interference factors as one product. The zero borders give $x_i = A_{z^{i3}}$ and $y_j = B_{z^{3j}}$, so this last product can be formed from the same pooled input factors. Subtracting it recovers our output:

$$C_{r,c} = \sum_{i,j\in[3]} W_{z^{ij}} - \Big(\sum_{i=1}^{3} A_{z^{i3}}\Big)\Big(\sum_{j=1}^{3} B_{z^{3j}}\Big). \tag{9}$$

The identity holds for every output line. Each uses nine grid-point products and one private correction; the grid-point products are computed only once for all lines through their points. Thus the full $R \times S$ block product uses

$$T = \underbrace{9^k}_{\text{shared}} + \underbrace{RS}_{\text{private}} = (k+9)9^{k-1}, \qquad \frac{T}{RS} = 1 + \frac{9}{k} \tag{10}$$

smaller matrix products instead of $4RS$. As k increases, the cost approaches one smaller product per output. We now show that much of this saving survives even if only a sparse set of entries is requested.

1.2 Recursion and Running Time

Following only requested entries. Apply the block identity to a product of dimensions $N \times D$ by $D \times M$. Divide the outer dimensions into R and S blocks and the middle dimension into four blocks. Every shared or private child is a product of dimensions $(N/R) \times (D/4)$ by $(D/4) \times (M/S)$.

Because the decoder (9) is an identity of matrices, one requested scalar entry needs only the corresponding entry from nine shared child products and one private child product. Recursively request these entries, merging identical requests to a shared child. Thus one entry of a pooled product is computed once, no matter how many outputs need it. At middle dimension one, the remaining requests are scalar multiplications, and the decoder reconstructs the answers on the way back up.

Measuring how much sharing survives. Consider a subproblem with z requested scalar entries among s possible output entries. The private children partition the output matrix, so together they have s possible positions and receive exactly z requests. A request can reach nine shared children, but there are only 9^k shared child matrices, each with $s/(RS)$ positions. Thus the shared children have a total of $(9/k)s$ possible positions and receive at most $9z$ requests:

	requests	possible positions
private children	z	s
shared children	$\leq 9z$	$(9/k)s$.

If we tracked requests alone, we would see a factor of ten per level; if we tracked only the number of child positions, we would miss the sparsity of the requests. The geometric mean captures both.

At a recursion level, let z_v and s_v denote the requests and possible positions of each subproblem v, and put

$$\Phi = \sum_v \sqrt{z_v s_v}.$$

Because $z_v \leq s_v$, this potential bounds the total number of requested entries. Moreover, Cauchy–Schwarz bounds the potential of any family of children by the geometric mean of its total requests and total possible positions. Hence the private children of one parent contribute at most $\sqrt{zs}$, and its shared children at most $\sqrt{9z \cdot (9s/k)} = (9/\sqrt{k})\sqrt{zs}$. Consequently, writing Φ_j for the potential at depth j,

$$\Phi_{j+1} \leq \left(1 + \frac{9}{\sqrt{k}}\right) \Phi_j, \qquad \Phi_0 = \sqrt{NMw}. \tag{11}$$

The factor tends to one as k grows, precisely because the potential captures the reuse that a naive request count would miss. With $t = \log_4 D$ levels, the leaf multiplications and the work of reconstructing requested entries take

$$\widetilde{O}\left(\sqrt{NMw}\left(1 + \frac{9}{\sqrt{k}}\right)^t\right).$$

Preparing the smaller matrices. The potential bounds the multiplication and reconstruction work, not the cost of constructing child inputs. Every child input entry is a signed sum of $O(k)$ parent entries. We simply prepare the inputs of *all* children, including those without requested entries; this cost will form the additive term in the theorem.

At depth j, there are at most T^j subproblems. Each has left input size $ND/(4R)^j$ and right input size $MD/(4S)^j$. Summing the work of forming their inputs over all levels gives

$$O_k\left(\sum_{j=1}^{t}\left[ND\left(\frac{T}{4R}\right)^j + MD\left(\frac{T}{4S}\right)^j\right]\right) = O_k\left(N\left(\frac{T}{R}\right)^t + M\left(\frac{T}{S}\right)^t\right). \tag{12}$$

By (10) and the definitions of R, S,

$$\frac{T}{R} = \left(1 + \frac{9}{k}\right)3^{k-1}, \qquad \frac{T}{S} = (k+9)3^{k-1}.$$

Thus, with

$$\kappa = \left\lceil \log_4\left((k+9)3^{k-1}\right)\right\rceil, \tag{13}$$

preparing all child inputs costs $O_k((N+M)D^\kappa)$.

Choosing the parameters. The two bounds explain the parameter choice: larger k reduces the cost of following requests but increases the cost of preparing the child matrices. Choose

$$k = \max\left\{4, \left\lceil\left(\frac{9}{4^\tau - 1}\right)^2\right\rceil\right\}.$$

Then $1 + 9/\sqrt{k} \le 4^\tau$ and $\kappa = O(k) = O(\tau^{-2})$. Since $D = 4^t$, the cost of following requests is $\widetilde{O}(D^\tau \sqrt{NMw})$, while preparing the inputs costs $O_k((N+M)D^\kappa)$.

Finally, pad d to a power D of four and the outer dimensions to multiples of R^t and S^t. Since $R^t, S^t \le D^\kappa = O_k(d^\kappa)$, the assumption $d^\kappa \le \min\{n, m\}$ keeps the padded dimensions at $N = O_k(n)$ and $M = O_k(m)$. Each level introduces only $O(k)$ additions per entry, so the intermediate integers still have $O_k(\log(n+m))$ bits. Combining the two costs proves (1); the case $d = 1$ is immediate.

Improved constants. The same idea works with $q \times q$ checksum tables, starting from $(q-1)^2$ products. The grid then has side length q^2, and each recursion level reduces the middle dimension by a factor of $(q-1)^2$. Taking $q = 4$ instead of $q = 3$ slightly improves the parameters: in the regime $n = m = N$ and $w = N^2/\sqrt{d}$, optimizing them increases the exponent saving from approximately 0.0001816 to 0.0002050. We use $q = 3$ for simpler notation.

2 All-Pairs Shortest Paths (APSP)

We reduce APSP directly to selected-entry multiplication. For a min-plus product, we first examine a common random sample of candidate indices. Only a few candidates can improve the sampled minimum in expectation. We find them by dividing the indices into blocks, using selected products to identify blocks that contain an improvement, and inspecting only those blocks. The only additional ingredient is a simple way to express comparisons as Boolean matrix products with a sparse correction.

We use the following bound from Theorem 1, for some constant $\kappa \ge 1$: if $D^\kappa \le n$, any w specified entries of XY, where $X \in \mathbb{Z}^{n \times D}$ and $Y \in \mathbb{Z}^{D \times n}$ have polynomially bounded entries, can be computed in time

$$\widetilde{O}\big(nD^\kappa + nD^{1/8}\sqrt{w}\big). \tag{14}$$

We will use it only for Boolean matrices. In particular, the reduction places no restriction on the magnitudes of the original edge weights.

Theorem 2. *There is a Las Vegas algorithm for APSP in directed graphs with arbitrary real edge weights and no negative cycles, with expected running time*

$$\widetilde{O}\big(n^{3-1/(16\kappa+2)}\big).$$

Proof. It suffices to compute a min-plus product. We begin with a thin one,

$$C_{ij} = \min_{k \in [d]} \{A_{ik} + B_{kj}\}, \qquad A \in \mathbb{R}^{n \times d}, \quad B \in \mathbb{R}^{d \times n}.$$

Fix an integer $t \ge 2$ and, for now, assume $d = t^9 \le n$. We take a sample of size $r = d/t = t^8$ and partition the candidate indices into blocks of size $b = r/t = t^7$. The point of these choices is that both sampling and inspecting the few relevant blocks will cost $O(n^2 r)$; the choice $d = t^9$ also makes the selected-product work fit this bound.

Sampling and recovery. Choose a uniformly random r-element set $S \subseteq [d]$. For each output pair (i, j), compute its best sampled candidate

$$p_{ij} \in \arg\min_{k \in S} \{A_{ik} + B_{kj}\},$$

using $O(n^2 r)$ time in total. All pairs use the *same* sample, so there are at most r distinct indices among the p_{ij}.

Call k an *improvement* for (i, j) if $A_{ik} + B_{kj} < A_{i,p_{ij}} + B_{p_{ij},j}$, and let q_{ij} count its improvements. Order the d candidates by value, breaking ties arbitrarily. The r sampled positions divide the remaining $d - r$ positions into $r + 1$ gaps, all with the same expected size, $(d-r)/(r+1)$. Every improvement precedes the first sampled position, so

$$\mathbb{E}[q_{ij}] \le \frac{d-r}{r+1} < \frac{d}{r} = t, \qquad \mathbb{E}\sum_{i,j} q_{ij} \le n^2 t. \tag{15}$$

Partition $[d]$ into $d/b = t^2$ fixed blocks K of size b. A block is *active* for (i, j) when it contains an improvement. Once we know the active blocks, we inspect every candidate in each such block and take the minimum together with the sampled candidate. This gives the exact minimum, since any candidate that could beat it lies in an active block. Each active block contains at least one improvement, so (15) bounds the expected inspection time by

$$O\left(b\,\mathbb{E}\sum_{i,j} q_{ij}\right) = O(n^2 bt) = O(n^2 r). \tag{16}$$

We only need an efficient way to recognize active blocks.

Compressing comparisons. Fix a possible sampled candidate $p \in S$ and an index $k \in [d]$. For this p, k, the test that k improves p separates as

$$A_{ik} + B_{kj} < A_{ip} + B_{pj} \quad \Longleftrightarrow \quad \underbrace{A_{ik} - A_{ip}}_{x_i} < \underbrace{B_{pj} - B_{kj}}_{y_j}. \tag{17}$$

Thus we need the comparison matrix $M^{p,k}_{ij} = [x_i < y_j]$, formed from one list of row values and one list of column values.

Sort the $2n$ tagged values x_i and y_j in nondecreasing order, placing every y-item before every x-item when their values are equal. Partition the resulting order into d consecutive buckets of size at most $s = \lceil 2n/d \rceil$. Let α_i and β_j denote the bucket indices of x_i and y_j, respectively.

The bucket indices decide the comparison unless the two items are in the same bucket. Indeed, if $\alpha_i < \beta_j$, then x_i precedes y_j in the sorted order. Their values must be *strictly* ordered: if they were equal, the tie rule would put y_j first. Conversely, if $\alpha_i > \beta_j$, then y_j precedes x_i, so $x_i \geq y_j$ and the comparison $x_i < y_j$ is false. Here equality is possible, even across different buckets; it causes no difficulty. Thus

$$[x_i < y_j] = [\alpha_i < \beta_j] + [\alpha_i = \beta_j]\,[x_i < y_j]. \tag{18}$$

Only comparisons within a bucket need to be checked explicitly.

To express the first term as a matrix product, define the Boolean matrices

$$U^{p,k}_{i,h} = [\alpha_i = h], \qquad V^{p,k}_{h,j} = [h < \beta_j], \qquad h \in [d].$$

Each row of $U^{p,k}$ has exactly one nonzero, so $(U^{p,k}V^{p,k})_{ij} = [\alpha_i < \beta_j]$. Defining

$$E^{p,k}_{ij} = [\alpha_i = \beta_j]\,[x_i < y_j],$$

we obtain the identity

$$M^{p,k} = U^{p,k}V^{p,k} + E^{p,k}. \tag{19}$$

The two terms have disjoint support. Moreover, each row item shares its bucket with at most s column items, giving

$$\operatorname{nnz}(E^{p,k}) \leq ns = O(n^2/d). \tag{20}$$

We can find these nonzeros simply by examining row–column pairs within each bucket. Thus an entire matrix of strict comparisons is represented by a Boolean product with inner dimension d, together with $O(n^2/d)$ explicitly listed corrections.

Detecting active blocks. Recall that a block K is active for (i, j) when some $k \in K$ improves its sampled candidate p_{ij}. Fix a possible sampled candidate $p \in S$ and a block K. Concatenate the comparison factorizations over $k \in K$:

$$X^{p,K} = \left[\,U^{p,k}\,\right]_{k\in K}, \qquad Y^{p,K} = \left[V^{p,k}\right]_{k\in K}.$$

These Boolean matrices have inner dimension

$$D = |K|d = bd = t^{16}.$$

Summing (19) over the block gives

$$\sum_{k\in K}[A_{ik}+B_{kj}<A_{ip}+B_{pj}]=(X^{p,K}Y^{p,K})_{ij}+\sum_{k\in K}E^{p,k}_{ij}. \tag{21}$$

The left-hand side counts the improvements in K. Since all terms on the right are nonnegative, K is active for a pair with $p_{ij}=p$ precisely when either the selected product entry or one of the explicit corrections is positive.

The corrections are inexpensive to process. By (20), over all $p\in S$ and $k\in[d]$ there are only

$$O\left(rd\cdot\frac{n^2}{d}\right)=O(n^2r)$$

nonzero corrections. For each, we mark the block containing k as active for (i,j) if $p_{ij}=p$. For the product contribution, write

$$W_p=\{(i,j):p_{ij}=p\},\qquad w_p=|W_p|.$$

For every $p\in S$ and block K, compute just the w_p entries of $X^{p,K}Y^{p,K}$ indexed by W_p, and mark the blocks with positive answers. Equation (21) shows that these are exactly the active blocks. We then scan them as described above; in particular, no candidate capable of improving the sampled minimum is missed.

Running time. The sets W_p partition all n^2 output pairs, and hence Cauchy–Schwarz gives

$$\sum_{p\in S}\sqrt{w_p}\le\sqrt{r\sum_{p\in S}w_p}=n\sqrt{r}=nt^4.$$

There are $r=t^8$ sampled candidates and t^2 blocks. For each p,K, the selected-entry bound (14) costs $\widetilde{O}(nD^\kappa+nD^{1/8}\sqrt{w_p})$. Summing and using $D=t^{16}$ yields

$$\widetilde{O}\left(nrt^2D^\kappa+nt^2D^{1/8}\sum_{p\in S}\sqrt{w_p}\right)=\widetilde{O}\left(nt^{16\kappa+10}+n^2t^8\right).$$

This also covers all other work: sampling and the expected scans cost $O(n^2r)=O(n^2t^8)$; sorting the rd lists costs $\widetilde{O}(nrd)$; and processing their sparse corrections costs $O(n^2r)$. Constructing the Boolean inputs takes $O(nrt^2D)=O(nt^{26})$, within the first term since $\kappa\ge1$. Here we use $d=t^9\le n$ to absorb the sorting cost. Therefore a thin product takes expected time

$$\widetilde{O}\left(nt^{16\kappa+10}+n^2t^8\right). \tag{22}$$

For square min-plus multiplication, partition the n candidate indices into $O(n/d)$ groups, compute the corresponding thin products, and take their entrywise minimum. This takes expected time

$$\widetilde{O}\left(n^2t^{16\kappa+1}+\frac{n^3}{t}\right).$$

Choosing $t=\Theta(n^{1/(16\kappa+2)})$, with a sufficiently small constant, balances these terms and ensures both $D^\kappa\le n$ and $d\le n$. We obtain

$$\widetilde{O}\left(n^{3-1/(16\kappa+2)}\right)$$

expected time for a square min-plus product.

Finally, APSP follows from $O(\log n)$ repeated min-plus squarings. To handle missing edges using finite matrix entries, let $H=\max\{1,\max_e|w_e|\}$, replace each missing edge by $M=(4n+1)H$, and set the diagonal to zero. Every finite shortest path has a simple representative of at most $n-1$ edges and weight at most nH. On the other hand, every walk of at most $2n$ edges that uses a replacement edge has weight greater than nH. The squarings consider walks of at most $2n$ edges, so we report values exceeding nH as infinite. □

3 Integer 3SUM

Given three sets A, B, C of at most n polynomially bounded integers, we want to decide whether $c \in A + B$ for every $c \in C$. The idea is to hash integers to binary masks. When the masks of a and b are disjoint, their numerical sum is their union, so subset convolution detects $a + b = c$. A second-moment test excludes false matches caused by modular hashing.

Theorem 3 (Randomized integer 3SUM)**.** *For $A, B, C \subseteq [-n^{O(1)}, n^{O(1)}]$ with $|A|, |B|, |C| \le n$, whether $c \in A + B$ can be decided for every $c \in C$ by a one-sided Monte Carlo algorithm in expected $O(n^{1.965})$ word operations, with total error probability at most n^{-2}.*

Trimmed subset convolution. For functions $F, G : 2^{[d]} \to R$ supported on sets of size t, define

$$(F * G)(Z) = \sum_{X \subseteq Z} F(X)G(Z \setminus X), \qquad |Z| = 2t.$$

Here R is any fixed commutative ring. We use $\widetilde{O}$ to hide polylogarithmic factors. Write $\binom{u}{\le v} = \sum_{j \le v} \binom{u}{j}$ and $\binom{u}{\ge v} = \sum_{j \ge v} \binom{u}{j}$.

Lemma 4 (Selected-entry subset convolution)**.** *For $t \le s \le 2t$, we can preprocess F, G in $O\big(d(n + \binom{d}{\le s})\big)$ ring operations and subsequently compute $(F * G)(Z)$ for any $|Z| = 2t$ in $O\big(d\binom{2t}{\ge s}\big)$ ring operations.*

Proof. Write $\mathcal{Z}U(X) = \sum_{Y \subseteq X} U(Y)$ and $\mathcal{M}U(X) = \sum_{Y \subseteq X} (-1)^{|X \setminus Y|} U(Y)$ for the zeta and inverse zeta transforms. Let $H(Z) = \sum_{X \cup Y = Z} F(X)G(Y)$ be the covering product. Then

$$\mathcal{Z}H(X) = \mathcal{Z}F(X)\, \mathcal{Z}G(X), \qquad H = \mathcal{M}(\mathcal{Z}F \cdot \mathcal{Z}G).$$

For $|Z| = 2t$, the condition $X \cup Y = Z$ and $|X| = |Y| = t$ forces $X \cap Y = \varnothing$; hence $H(Z) = (F * G)(Z)$.

Both transforms use the same dynamic program. Starting from $U_0(X) = U(X)$, for $i = 1, \dots, d$ compute

$$U_i(X) = U_{i-1}(X) + \varepsilon \mathbf{1}_{i \in X} U_{i-1}(X \setminus \{i\}), \qquad \varepsilon = +1 \text{ or } -1. \tag{23}$$

The final array is $\mathcal{Z}U$ for $\varepsilon = +1$ and $\mathcal{M}U$ for $\varepsilon = -1$. Precompute all intermediate states of the two forward transforms and the inverse transform on $|X| \le s$. This uses $O(d\binom{d}{\le s})$ operations. For a query Z, continue the same three DPs only on $X \subseteq Z$ with $|X| > s$, using the saved states at $|X| = s$ as boundary values. Dependencies in (23) only delete elements, so no other states are needed. There are at most $\binom{2t}{\ge s}$ such states per DP stage. This proves the claimed costs. □

One convolution also verifies the sums. Use the ring of integer triples with coordinatewise addition and multiplication

$$(u_0, u_1, u_2)(v_0, v_1, v_2) = (u_0 v_0,\ u_1 v_0 + u_0 v_1,\ u_2 v_0 + 2u_1 v_1 + u_0 v_2).$$

This is polynomial multiplication truncated after degree two, with the quadratic coefficient stored twice, so it is associative. Assign an integer a the weight $w(a) = (1, a, a^2)$. Then $w(a)w(b) = (1, a + b, (a + b)^2)$. Thus a subset-convolution query returns $M = (M_0, M_1, M_2)$, the count and first two moments of all contributing pairs. We accept a target c precisely when

$$M_0 > 0, \qquad M_2 - 2cM_1 + c^2 M_0 = 0. \tag{24}$$

Indeed, the second expression is $\sum_{(a,b) \text{ contributing}} (a + b - c)^2$, so the test cannot accept a false target. If several numbers hash to the same mask, simply add their weights; no collision handling is required.

Random hashing. Let lg denote the base-two logarithm, and set

$$t = \lfloor 0.2 \lg n \rfloor, \qquad s = \lfloor 0.367 \lg n \rfloor, \qquad P = 2^d = \Theta(n 2^t \log n), \quad T = \binom{d}{2t}, \quad K = \binom{2t}{t}.$$

Choose the constant in P sufficiently large. In each trial, independently pick a uniform prime $p \in (P, 2P]$, a uniform $\xi \in \mathbb{F}_p^*$, and uniform $\alpha, \beta \in \mathbb{F}_p$. Write

$$x(a) = (\xi a + \alpha) \bmod p, \qquad y(b) = (\xi b + \beta) \bmod p, \qquad z(c) = (\xi c + \alpha + \beta) \bmod p. \tag{25}$$

For $0 \le u < P$, let $\pi(u) \subseteq [d]$ be its one-bit positions, and write $\mathrm{val}(X) = \sum_{i \in X} 2^{i-1}$ for the inverse encoding. In particular, if $a + b = c$ and $\pi(x(a)) \cap \pi(y(b)) = \varnothing$, then $z(c) = x(a) + y(b) < P$ and $\pi(z(c)) = \pi(x(a)) \cup \pi(y(b))$.

Algorithm 1 Randomized 3SUM with trimmed subset convolution

Input: Sets A, B, C of at most n polynomially bounded integers

Set t, s, d, P, T, K as above and $R \leftarrow \lceil c_0 P^2 \log n / (TK) \rceil$ for a sufficiently large constant c_0
Mark every $c \in C$ as unanswered
for $r = 1, \dots, R$ **do**
    Independently draw p, ξ, α, β and define x, y, z by (25)
    Initialize sparse maps F, G with default value $(0, 0, 0)$
    for $a \in A$ **do**
        if $x(a) < P$ and $|\pi(x(a))| = t$ **then**
            $F(\pi(x(a))) \leftarrow F(\pi(x(a))) + w(a)$
    for $b \in B$ **do**
        if $y(b) < P$ and $|\pi(y(b))| = t$ **then**
            $G(\pi(y(b))) \leftarrow G(\pi(y(b))) + w(b)$
    Build the trimmed convolution index for F, G at cutoff s
    for each unanswered $c \in C$ with $z(c) < P$ and $|\pi(z(c))| = 2t$ **do**
        $(M_0, M_1, M_2) \leftarrow (F * G)(\pi(z(c)))$ using the index
        if $M_0 > 0$ and $M_2 - 2cM_1 + c^2 M_0 = 0$ **then**
            Mark c as answered *yes*
Answer *no* for every remaining unanswered c

Lemma 5 (A fixed witness survives)**.** *For any $a \in A$, $b \in B$, and $c \in C$ with $a + b = c$, one trial of Algorithm 1 accepts c with probability $\Omega(TK/P^2)$.*

Proof. Fix a witness $a + b = c$. We first bound two ways in which a prime $p \in (P, 2P]$ can be unfavorable.

Call p *bad* if some $a' \in A \setminus \{a\}$ satisfies $a' \equiv a \pmod p$, or some $b' \in B \setminus \{b\}$ satisfies $b' \equiv b \pmod p$. Let D_p be the indicator of this event, and let

$$N_p = \#\{(a', b') \in A \times B : a' + b' \neq c, \quad a' + b' \equiv c \pmod p\}.$$

Thus N_p counts incorrect pairs whose sums happen to agree with c modulo p.

Every nonzero integer of polynomial magnitude has only $O(1)$ prime divisors larger than $P = n^{1.2+o(1)}$. Since there are $\Omega(P/\log n)$ primes in $(P, 2P]$, a uniformly chosen prime divides any fixed nonzero difference with probability $O(\log n / P)$. A union bound therefore gives

$$\mathbb{E}_p D_p = O\left(\frac{n \log n}{P}\right), \qquad \mathbb{E}_p N_p = O\left(\frac{n^2 \log n}{P}\right). \tag{26}$$

Now fix a prime p and disjoint masks $X, Y \subseteq [d]$ of size t. The event

$$x(a) = \mathrm{val}(X), \qquad y(b) = \mathrm{val}(Y)$$

has probability exactly p^{-2}: for every choice of ξ, there is exactly one choice of α, β realizing these two equalities. In particular, ξ remains uniform in $\mathbb{F}_p^*$ after conditioning on this event.

Under the event, the witness contributes to the query $Z = X \cup Y$. Indeed, since X and Y are disjoint,

$$z(c) = x(a) + y(b) = \mathrm{val}(Z) < P < p.$$

We only need to ensure that no incorrect pair contributes to the same query, since then every contributing pair is a genuine solution and the moment test accepts.

Suppose p is not bad. If an incorrect pair (a', b') contributes to Z, necessarily

$$a' + b' \equiv c \pmod{p}.$$

Moreover, $a' \not\equiv a \pmod{p}$: otherwise $a' = a$ by the choice of p, and the displayed congruence would imply $b' \equiv b \pmod{p}$, hence $b' = b$, a contradiction.

Consequently,

$$x(a') = x(a) + \xi(a' - a) \pmod{p}$$

is uniform over $\mathbb{F}_p \setminus \{x(a)\}$. To contribute to Z, its mask must be one of the $K = \binom{2t}{t}$ subsets of Z of size t. Thus each incorrect pair counted by N_p contributes with conditional probability at most $K/(p-1) \le K/P$. By a union bound, the probability of any incorrect contribution is at most KN_p/P.

There are exactly TK ordered pairs of disjoint t-element masks (X, Y), and their witness-hash events are disjoint. It follows that

$$\Pr(\text{accept } c \mid p) \ge \frac{TK}{p^2}\left(1 - D_p - \frac{KN_p}{P}\right).$$

The bound remains valid when the right-hand side is negative.

Finally, average over the uniformly chosen prime. Since $P < p \le 2P$, (26) gives

$$\begin{aligned}\Pr(\text{accept } c) &\ge \frac{TK}{4P^2} - \frac{TK}{P^2}\left(\mathbb{E}_p D_p + \frac{K}{P}\mathbb{E}_p N_p\right) \\ &\ge \frac{TK}{P^2}\left(\frac{1}{4} - O\left(\frac{n\log n}{P} + \frac{n^2 K \log n}{P^2}\right)\right).\end{aligned}$$

Using $P = \Theta(n2^t \log n)$ and $K \le 4^t$, the error term is

$$O\left(2^{-t} + \frac{1}{\log n}\right) = o(1).$$

Hence the success probability is $\Omega(TK/P^2)$. □

Proof of Theorem 3. The moment test never accepts a false target: its second expression is

$$M_2 - 2cM_1 + c^2 M_0 = \sum_{(a,b)\ \text{contributing}} (a + b - c)^2,$$

which vanishes for a nonempty collection of contributing pairs exactly when every pair satisfies $a + b = c$.

For a true target, Lemma 5 gives success probability $\Omega(TK/P^2)$ in each trial. The trials are independent, so for a sufficiently large constant c_0,

$$R = \left\lceil \frac{c_0 P^2 \log n}{TK} \right\rceil$$

makes the probability of missing any fixed true target at most n^{-3}. A union bound over the at most n targets gives total error probability at most n^{-2}.

It remains to bound the expected running time. Write

$$H = \binom{d}{\le s}, \qquad Q = \binom{2t}{\ge s}.$$

By Lemma 4, building the index costs $\widetilde{O}(n + H)$ per trial, and each eligible target costs $\widetilde{O}(Q)$ to query.

For any fixed p and c, the hash $z(c)$ is uniform in $\mathbb{F}_p$. Exactly $T = \binom{d}{2t}$ integers below P have binary weight $2t$, so the probability that c is eligible is $T/p \le T/P$. Thus the expected number of eligible targets per trial is at most nT/P, and the total expected time is

$$\widetilde{O}\left(R\left(n + H + \frac{nTQ}{P}\right)\right).$$

Since $K = \Theta(4^t/\sqrt{t})$ and $P = \Theta(n2^t \log n)$, we have $R = \widetilde{O}(n^2/T)$. Substituting yields

$$\widetilde{O}\left(\frac{n^3}{T} + \frac{n^2 H}{T} + \frac{n^3 Q}{P}\right). \tag{27}$$

For $t = \lfloor 0.2 \lg n \rfloor$, $s = \lfloor 0.367 \lg n \rfloor$, and $d = 1.2 \lg n + O(\log \log n)$, standard binomial-entropy bounds give

$$T = n^{1.101955+o(1)}, \quad H = n^{1.065966+o(1)},$$
$$Q = n^{0.164371+o(1)}, \quad P = n^{1.2+o(1)}.$$

The three terms in (27) consequently have exponents 1.898045, 1.964011, and 1.964371, respectively, up to polylogarithmic factors. Thus the expected running time is $O(n^{1.965})$.

All ring elements have polynomial magnitude, so each arithmetic operation takes $O(1)$ machine-word operations. □

References

[1] Timothy M. Chan, Reina Itakura, Zhiyuan Ma, and Yuancheng Yu. *3SUM Made Simpler (Without Matrix Multiplication), k-SUM Hypothesis Refuted for Odd k, and Real 3SUM/APSP Derandomized*. 2026. arXiv:2610.42012.

[2] Sankeerth Rao Karingula and Shachar Lovett. *A simplified proof of polynomial improvements for 3SUM and APSP*. 2026. arXiv:2610.15663v1.